Lecture 1: Vectors Through a Traffic-Control Problem

AI 501: Mathematics for Artificial Intelligence

Fall 2026

Abstract

We develop vectors through one problem: choosing a traffic-signal plan at a four-way intersection. A traffic state must be represented, compared with previous states, used to predict delay, and decomposed into recurring patterns. Each task introduces a basic idea of linear algebra and its role in AI.

Contents

1 From AI applications to one mathematical problem **1**
1.1 The intersection 2

2 One traffic state is a vector **2**
2.1 Definition, size, and interpretations 2
2.2 Geometric and application pictures 3

3 Changes, combinations, and a first prediction **3**
3.1 Addition and scalar multiplication 3
3.2 A hierarchy of combinations 4
3.3 The inner product: score, alignment, and projection 4

4 Size, error, and similarity **5**
4.1 Norms and distances 5
4.2 Prediction error, MAE, and RMSE 6
4.3 Angles, cosine similarity, and projection 6
4.4 Mean, RMS, standard deviation, correlation, and standardization 7

5 From combinations to spaces of possible patterns **8**
5.1 Span and linear independence 8
5.2 Basis and dimension 8
5.3 Orthonormal bases and coefficients 9
5.4 Gram–Schmidt orthogonalization 9

6 Vector spaces and subspaces **10**

7 Putting the Ideas Together **11**

---

1 From AI applications to one mathematical problem

AI systems convert observations into numerical representations, use a model, and then make a decision. Images, sound, transactions, and medical readings look different, but each can be represented by a list of numbers. This course studies the mathematics of those representations.

1.1 The intersection

At the beginning of each 90-second cycle, a signal controller chooses green times for a four-way intersection. Cameras estimate the queues on the north, east, south, and west approaches.

Approach	North	East	South	West
Vehicles waiting	30	18	25	12

The controller has records of earlier queues, signal plans, and resulting delays. We will ask how to store a state, compare cycles, predict delay, and describe the recurring patterns in many cycles.

2 One traffic state is a vector

Write the current state as

$$\mathbf{x} = \begin{bmatrix} 30 \\ 18 \\ 25 \\ 12 \end{bmatrix} = \begin{bmatrix} x_N \\ x_E \\ x_S \\ x_W \end{bmatrix} \in \mathbb{R}^4.$$

The order matters: the entries correspond, in order, to northbound, eastbound, southbound, and westbound queues.

2.1 Definition, size, and interpretations

An *n-vector* is an ordered finite list of n numbers. We use real vectors, written as columns,

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n.$$

The number n is the vector's *size* or *dimension*, and x_i is its i-th *entry*. Equality requires equal sizes and equal corresponding entries. Here dimension refers informally to the number of coordinates; it need not be a physical spatial dimension. Later, $\dim(V)$ will denote the dimension of a vector space, defined as the number of vectors in any basis.

The same mathematical object has different interpretations in different contexts. The vector $\mathbf{x} = (30, 18, 25, 12)^\mathsf{T}$ can be

1. an ordered list of four queue counts;
2. a point (or state) in a four-dimensional traffic-state space; or
3. a displacement or direction when it is a difference.

For example, if a later state is $\mathbf{x}'$, then $\Delta\mathbf{x} = \mathbf{x}' - \mathbf{x}$ naturally means the change from one traffic state to another. In AI, an individual sample is commonly a point in a high-dimensional feature space; the same vector may be interpreted as a location, a state, or a change depending on the question being asked.

Three vectors recur often. The *zero vector* $\mathbf{0}$ has all entries zero, and the *one vector* $\mathbf{1}$ has all entries one. The standard unit vector $\mathbf{e}_i$ has a one in entry i and zeros elsewhere. In traffic notation,

$$\mathbf{e}_N = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_E = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_S = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_W = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

Over $\mathbb{R}^n$, there is no smallest nonzero change: $0.1\mathbf{e}_N$ and $0.001\mathbf{e}_N$ are both possible. But when queues are counted in whole vehicles, $\mathbf{e}_N$ means a one-vehicle change in the northbound component only. The unit vectors also give the component-wise representation

$$\mathbf{x} = 30\mathbf{e}_N + 18\mathbf{e}_E + 25\mathbf{e}_S + 12\mathbf{e}_W.$$

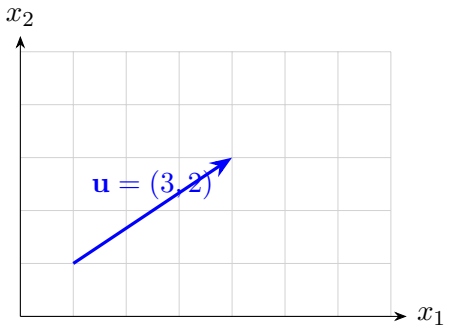
This idea will return when we study span and basis.

A vector is *k-sparse* if it has at most k nonzero entries. For instance, $(0, 5, 0, 0)^\mathsf{T}$ is 1-sparse. Bag-of-words vectors, one-hot categorical features, and other high-dimensional data are often sparse because only a few features are active for one observation.

Check yourself. If only the eastbound queue increases by four vehicles, express the change using the standard basis.
Answer. $4\mathbf{e}_E$.

2.2 Geometric and application pictures

In $\mathbb{R}^2$, $(3, 2)^\mathsf{T}$ can be drawn as a displacement: three units right and two units up.



The traffic vector does not live in a useful physical plane, but length, angle, direction, and projection still provide useful language.

Vectors can represent locations; displacements, velocities, or accelerations; quantities of products; values across a population (for example, patients' blood pressures); RGB colours; image pixels; time series such as monthly rainfall; and audio samples. In machine learning, a *feature vector* collects measured attributes of one object. The entries must always have a fixed, documented meaning.

3 Changes, combinations, and a first prediction

Twenty minutes later, suppose the camera reports

$$\mathbf{x}' = \begin{bmatrix} 34 \\ 15 \\ 20 \\ 14 \end{bmatrix}.$$

3.1 Addition and scalar multiplication

For $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ and scalar $a \in \mathbb{R}$, define

$$(\mathbf{u} + \mathbf{v})_i = u_i + v_i, \qquad (a\mathbf{u})_i = au_i, \qquad i = 1, \ldots, n.$$

Thus subtraction is $\mathbf{u} - \mathbf{v} = \mathbf{u} + (-1)\mathbf{v}$, and

$$\Delta\mathbf{x} = \mathbf{x}' - \mathbf{x} = \begin{bmatrix} 4 \\ -3 \\ -5 \\ 2 \end{bmatrix}.$$

The northbound queue grew by four; the southbound queue shrank by five. Addition and scaling obey familiar rules such as commutativity of addition and the distributive laws $a(\mathbf{u}+\mathbf{v}) = a\mathbf{u}+a\mathbf{v}$ and $(a+b)\mathbf{u} = a\mathbf{u} + b\mathbf{u}$.

3.2 A hierarchy of combinations

We can create a new vector from existing vectors by

$$\mathbf{y} = \sum_{i=1}^{k} c_i \mathbf{v}_i.$$

This is a *linear combination* when the real coefficients c_i have no restriction. It is an *affine combination* when $\sum_i c_i = 1$. Affine combinations preserve the affine geometry of points and are the natural form for interpolation. It is a *convex combination* when, in addition, $c_i \geq 0$ for every i; this is a weighted average. Hence

$$\boxed{\text{convex combinations} \subset \text{affine combinations} \subset \text{linear combinations.}}$$

For two recorded traffic states $\mathbf{x}_A$ and $\mathbf{x}_B$,

$$\mathbf{z} = \alpha\mathbf{x}_A + (1-\alpha)\mathbf{x}_B, \qquad 0 \leq \alpha \leq 1,$$

is a convex combination: an intermediate or blended traffic pattern. For example, with

$$\mathbf{x}_A = \begin{bmatrix} 32 \\ 16 \\ 28 \\ 10 \end{bmatrix}, \qquad \mathbf{x}_B = \begin{bmatrix} 20 \\ 30 \\ 18 \\ 22 \end{bmatrix},$$

the half-way blend is $(26, 23, 23, 16)^{\mathsf{T}}$. A coefficient outside $[0, 1]$ can still give an affine combination, but no longer a weighted average between the two states.

Check yourself. Is $2\mathbf{x}_A - \mathbf{x}_B$ linear, affine, convex, or more than one of these?
Answer. It is linear. Its coefficients sum to 1, so it is also affine; it is not convex because $-1 < 0$.

3.3 The inner product: score, alignment, and projection

For $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$, their *inner product* or *dot product* is

$$\mathbf{u}^{\mathsf{T}}\mathbf{v} = \sum_{i=1}^{n} u_i v_i.$$

For nonzero vectors with angle θ between them,

$$\mathbf{u}^{\mathsf{T}}\mathbf{v} = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \cos\theta.$$

This one identity gives several useful interpretations. The dot product is a weighted sum or score; it measures alignment; it underlies cosine similarity; it detects orthogonality; and it computes projections.

For a delay model, let learned feature weights and an intercept be

$$\mathbf{w} = \begin{bmatrix} 0.6 \\ 0.3 \\ 0.5 \\ 0.2 \end{bmatrix}, \qquad b = 4.$$

Then

$$\hat{y} = \mathbf{w}^\mathsf{T}\mathbf{x} + b = 0.6(30) + 0.3(18) + 0.5(25) + 0.2(12) + 4 = 42.3.$$

The vector $\mathbf{w}$ contains learned feature weights, and the dot product aggregates the features according to those weights. The same weighted-sum operation occurs inside a neuron: $z = \mathbf{w}^\mathsf{T}\mathbf{x} + b$.

The controller uses a dot product to score a traffic state; the same operation also scores compatibility in a recommender system. Matrix factorization represents a user by a latent feature vector $\mathbf{u}$ and an item by a latent feature vector $\mathbf{v}$, with predicted rating $\hat{r} = \mathbf{u}^\mathsf{T}\mathbf{v}$. A large dot product means that the learned user preferences and item characteristics align strongly.

In-class exercise. Let $\mathbf{x} = (x_0, \ldots, x_{99})^\mathsf{T}$ be a 100-vector, where x_i is the number of people aged i. Express the total population, the population aged 65 and above, and the average age using vector notation. This is the same idea as asking the controller for a total queue count or a count on selected approaches: a dot product queries a vector.
Solution. Let $\mathbf{a} = (0, 1, \ldots, 99)^\mathsf{T}$, and let $\mathbf{g}_{65+}$ have entries 0 for ages $0, \ldots, 64$ and 1 for ages $65, \ldots, 99$. Then

$$\text{total population} = \mathbf{1}^\mathsf{T}\mathbf{x}, \qquad \text{population aged } 65+ = \mathbf{g}_{65+}^\mathsf{T}\mathbf{x}, \qquad \text{average age} = \frac{\mathbf{a}^\mathsf{T}\mathbf{x}}{\mathbf{1}^\mathsf{T}\mathbf{x}},$$

provided the total population is nonzero. Returning to traffic, choosing an indicator vector with ones on north and south similarly gives their combined queue count.

4 Size, error, and similarity

4.1 Norms and distances

A *norm* measures vector size. A function $\|\cdot\|$ is a norm when, for all vectors $\mathbf{u}, \mathbf{v}$ and scalars a, it satisfies non-negativity $\|\mathbf{u}\| \geq 0$, definiteness $\|\mathbf{u}\| = 0 \Leftrightarrow \mathbf{u} = \mathbf{0}$, homogeneity $\|a\mathbf{u}\| = |a|\|\mathbf{u}\|$, and the triangle inequality $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$.

For $p \geq 1$, the p-norm is

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}.$$

Three cases are especially common:

$$\|\mathbf{x}\|_1 = \sum_i |x_i| \quad \text{(total absolute magnitude)}, \qquad \|\mathbf{x}\|_2 = \left(\sum_i x_i^2 \right)^{1/2} \quad \text{(Euclidean length)},$$

$$\|\mathbf{x}\|_\infty = \max_i |x_i| \quad \text{(largest absolute component)}.$$

For $\mathbf{a} = (3, -4, 12)^\mathsf{T}$,

$$\|\mathbf{a}\|_1 = 19, \qquad \|\mathbf{a}\|_2 = 13, \qquad \|\mathbf{a}\|_\infty = 12.$$

In AI, L_1 and L_2 norms later appear in regularization; L_2 gives ordinary Euclidean distance, while L_∞ reports a maximum error.

Distance is derived from a norm:

$$d_p(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_p.$$

Conceptually,

subtract $\longrightarrow$ displacement/error vector $\longrightarrow$ measure its size with a norm.

For the two traffic states above, $\Delta\mathbf{x} = (4, -3, -5, 2)^\mathsf{T}$, so

$$d_1(\mathbf{x}, \mathbf{x}') = 14, \qquad d_2(\mathbf{x}, \mathbf{x}') = \sqrt{54}, \qquad d_\infty(\mathbf{x}, \mathbf{x}') = 5.$$

Different norms encode different meanings of "close": total movement, Euclidean movement, or the largest one-approach change. This matters in nearest-neighbour methods and clustering.

4.2 Prediction error, MAE, and RMSE

For observed outcomes $\mathbf{y}$ and predictions $\hat{\mathbf{y}}$, use the residual (error) vector $\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}}$. For n test cases,

$$\text{MAE} = \frac{1}{n}\|\mathbf{r}\|_1, \qquad \text{RMSE} = \frac{1}{\sqrt{n}}\|\mathbf{r}\|_2.$$

Thus MAE and RMSE are simply scaled vector norms. MAE gives each absolute error equal weight; RMSE penalizes occasional large errors more strongly.

4.3 Angles, cosine similarity, and projection

Distance alone can be misleading when we care about a pattern rather than its absolute scale. If $\mathbf{h} = 2\mathbf{x}$, then $\mathbf{h}$ can be far from $\mathbf{x}$ in Euclidean distance, although it points in exactly the same direction. For nonzero $\mathbf{x}$ and $\mathbf{h}$,

$$\cos\theta = \frac{\mathbf{x}^\mathsf{T}\mathbf{h}}{\|\mathbf{x}\|_2\|\mathbf{h}\|_2}.$$

This *cosine similarity* is 1 for the same direction, 0 for orthogonal directions, and -1 for opposite directions. It compares patterns independently of magnitude, which is useful for embeddings, information retrieval, clustering, and recommendation.

Check yourself. If $\mathbf{y} = 3\mathbf{x}$ and $\mathbf{x} \neq 0$, what is their cosine similarity? Must they be close in Euclidean distance?
Answer. The cosine similarity is 1. They need not be close: $\|\mathbf{y} - \mathbf{x}\|_2 = 2\|\mathbf{x}\|_2$.

Projection makes "the part along a direction" precise. For nonzero $\mathbf{u}$,

$$\text{proj}_\mathbf{u}(\mathbf{x}) = \frac{\mathbf{x}^\mathsf{T}\mathbf{u}}{\mathbf{u}^\mathsf{T}\mathbf{u}}\mathbf{u}; \qquad \text{if } \|\mathbf{u}\|_2 = 1, \quad \text{proj}_\mathbf{u}(\mathbf{x}) = (\mathbf{x}^\mathsf{T}\mathbf{u})\mathbf{u}.$$

With $\mathbf{u} = (1, 0, 1, 0)^\mathsf{T}$, a north–south rush direction,

$$\text{proj}_\mathbf{u}(\mathbf{x}) = \frac{55}{2}\begin{bmatrix}1\\0\\1\\0\end{bmatrix} = \begin{bmatrix}27.5\\0\\27.5\\0\end{bmatrix}.$$

The residual $\mathbf{r} = \mathbf{x} - \text{proj}_\mathbf{u}(\mathbf{x}) = (2.5, 18, -2.5, 12)^\mathsf{T}$ satisfies $\mathbf{r}^\mathsf{T}\mathbf{u} = 0$: it has no component left along $\mathbf{u}$.

4.4 Mean, RMS, standard deviation, correlation, and standardization

For observations $\mathbf{x} = (x_1, \ldots, x_n)^\mathsf{T}$, define

$$\operatorname{avg}(\mathbf{x}) = \bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \operatorname{rms}(\mathbf{x}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2},$$

$$\operatorname{std}(\mathbf{x}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2}.$$

These use the population-style $1/n$ convention; the common sample standard deviation instead uses $1/(n-1)$. Expanding the square shows

$$\operatorname{rms}(\mathbf{x})^2 = \operatorname{avg}(\mathbf{x})^2 + \operatorname{std}(\mathbf{x})^2.$$

Indeed, $n^{-1}\sum_i (x_i - \bar{x})^2 = n^{-1}\sum_i x_i^2 - \bar{x}^2$. RMS therefore combines the typical level and variability, whereas standard deviation measures only variation around the mean.

Bias–variance trade-off. For the traffic controller, prediction *bias* is systematic error: the difference between its expected delay prediction and the true delay. *Variance* measures how much that prediction changes when the controller is trained on a different historical sample. A model that is too simple typically has high bias and underfits; a model that follows training cycles too closely typically has high variance and overfits. The trade-off is that reducing one can increase the other. Under the usual expectation over training sets/model randomness and outcome noise, the slide-level decomposition is

$$\boxed{\text{RMS Error}^2 = \text{Bias}^2 + \text{Variance} + \text{Noise}.}$$

Noise is irreducible variation in the outcome. This statistical decomposition is *not* the elementary identity $\operatorname{rms}(\mathbf{x})^2 = \operatorname{avg}(\mathbf{x})^2 + \operatorname{std}(\mathbf{x})^2$, which concerns the entries of one numerical vector.

For two observation vectors $\mathbf{x}$ and $\mathbf{y}$, form centered vectors

$$\widetilde{\mathbf{x}} = \mathbf{x} - \bar{x}\mathbf{1}, \qquad \widetilde{\mathbf{y}} = \mathbf{y} - \bar{y}\mathbf{1}.$$

Provided both centered vectors are nonzero, their correlation coefficient is

$$\rho_{xy} = \frac{\widetilde{\mathbf{x}}^\mathsf{T}\widetilde{\mathbf{y}}}{\|\widetilde{\mathbf{x}}\|_2 \|\widetilde{\mathbf{y}}\|_2}.$$

$$\boxed{\text{Correlation is cosine similarity between centered data vectors.}}$$

Centering removes typical levels, so correlation measures whether two features rise and fall together rather than whether their raw counts are large.

Standardization applies those two ideas to one feature:

$$z_i = \frac{x_i - \bar{x}}{s},$$

where s is its standard deviation (and $s > 0$). First centering translates observations so their mean is zero. Then scaling changes units so, under the same convention used for s, $\operatorname{avg}(\mathbf{z}) = 0$ and $\operatorname{std}(\mathbf{z}) = 1$. Scale-sensitive methods such as k-NN, k-means, SVMs, and gradient-based optimization can otherwise be distorted by features with dramatically different numerical scales.

5 From combinations to spaces of possible patterns

We have learned how to create one vector by combining existing vectors. The next question is: what complete collection of vectors can be generated this way?

5.1 Span and linear independence

The *span* of $\mathbf{v}_1, \ldots, \mathbf{v}_k$ is

$$\operatorname{span}(\mathbf{v}_1, \ldots, \mathbf{v}_k) = \left\{ \sum_{i=1}^{k} c_i \mathbf{v}_i : c_i \in \mathbb{R} \right\}.$$

If $\mathbf{p}_1, \mathbf{p}_2$ are two fundamental traffic patterns, an observed state in their span can be represented exactly using those patterns; an approximately represented state is close to that span. This is the basic idea behind low-dimensional representations.

Check yourself. Can two vectors span all of $\mathbb{R}^4$? Why or why not?
Answer. No. At most two independent directions are available, whereas $\mathbb{R}^4$ requires four independent directions.

Some generators are redundant. If one vector can already be constructed from the others, it contributes no genuinely new direction. For example, $\mathbf{v}_3 = \mathbf{v}_1 + \mathbf{v}_2$ is an obvious dependence. Formally, $\mathbf{v}_1, \ldots, \mathbf{v}_k$ are *linearly independent* if

$$c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0} \quad \Longrightarrow \quad c_1 = \cdots = c_k = 0.$$

Otherwise they are dependent. Independence means no pattern, or feature direction, is redundant with the others.

Check yourself. If $\mathbf{v}_3 = 2\mathbf{v}_1 - \mathbf{v}_2$, can the three vectors be independent?
Answer. No: $-2\mathbf{v}_1 + \mathbf{v}_2 + \mathbf{v}_3 = \mathbf{0}$ is a nontrivial relation.

5.2 Basis and dimension

A *basis* is enough but not redundant: it spans the space and is linearly independent,

$$\boxed{\text{basis} = \text{spanning} + \text{linear independence.}}$$

Every vector in the space has a unique representation in a basis. The standard unit vectors $\mathbf{e}_1, \ldots, \mathbf{e}_n$ are the standard basis of $\mathbb{R}^n$, closing the loop with the component representation at the start.

Dimension is the number of independent directions required to describe a space. Formally, $\dim(V)$ is the number of vectors in any basis of V. Do not confuse the number of entries in a vector with the dimension of the ambient space or a subspace: a vector in $\mathbb{R}^{100}$ can belong to a two-dimensional subspace.

Independence–Dimension Inequality. The controller's traffic states live in $\mathbb{R}^4$, so at most four traffic patterns can be linearly independent. More generally, in a d-dimensional vector space, any linearly independent set contains at most d vectors. Dually, any spanning set must contain at least d vectors. The following two-dimensional class example makes the same fact visible: three vectors in $\mathbb{R}^2$ cannot all be linearly independent. With

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad \mathbf{v}_3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

we have $\mathbf{v}_3 = \mathbf{v}_1 + \mathbf{v}_2$.

For example,

$$S = \operatorname{span}\left(\begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} \right) \subset \mathbb{R}^3.$$

The two generators are independent, so $\dim(S) = 2$, although every vector in S has three entries.

5.3 Orthonormal bases and coefficients

Bases need not be geometrically convenient. Two vectors are *orthogonal* when $\mathbf{u}^\mathsf{T}\mathbf{v} = 0$. A nonzero vector is normalized by $\hat{\mathbf{u}} = \mathbf{u}/\|\mathbf{u}\|_2$. A collection is *orthonormal* when its vectors are mutually orthogonal and have unit length; equivalently, $\mathbf{q}_i^\mathsf{T}\mathbf{q}_j = \delta_{ij}$.

If $\mathbf{x} = \sum_i c_i \mathbf{q}_i$ in an orthonormal basis, take the dot product with $\mathbf{q}_j$:

$$\mathbf{q}_j^\mathsf{T}\mathbf{x} = \sum_i c_i \mathbf{q}_j^\mathsf{T}\mathbf{q}_i = c_j.$$

Thus $c_j = \mathbf{q}_j^\mathsf{T}\mathbf{x}$. Orthonormal bases make coordinates and projections especially easy to compute.

Before returning to traffic patterns, use the standard orthonormal basis of $\mathbb{R}^3$ for a small numerical illustration. Take $\mathbf{x} = (3, 4, 0)^\mathsf{T}$. Since $\mathbf{e}_1^\mathsf{T}\mathbf{x} = 3$, $\mathbf{e}_2^\mathsf{T}\mathbf{x} = 4$, and $\mathbf{e}_3^\mathsf{T}\mathbf{x} = 0$,

$$\mathbf{x} = 3\mathbf{e}_1 + 4\mathbf{e}_2 + 0\mathbf{e}_3.$$

Exactly the same dot products recover the strengths of orthonormal traffic patterns in a traffic state.

Check yourself. If two unit vectors have dot product zero, what can we conclude?
Answer. They are orthonormal (and, in particular, orthogonal).

5.4 Gram–Schmidt orthogonalization

Suppose we have independent directions that overlap. Can we replace them by orthonormal directions spanning the same space? Gram–Schmidt does exactly this. Projection identifies the part already explained by an earlier direction; the algorithm repeatedly removes those parts.

For independent $\mathbf{v}_1, \ldots, \mathbf{v}_k$, the two stages are

$$\text{Orthogonalization:} \quad \mathbf{u}_1 = \mathbf{v}_1, \qquad \mathbf{u}_j = \mathbf{v}_j - \sum_{i=1}^{j-1} \operatorname{proj}_{\mathbf{u}_i}(\mathbf{v}_j),$$

$$\text{Normalization:} \quad \mathbf{q}_j = \frac{\mathbf{u}_j}{\|\mathbf{u}_j\|_2}.$$

Each subtraction removes from the new vector every component already explained by the previously constructed directions.

For transparent arithmetic, the class example uses two-dimensional directions rather than four traffic components:

$$\mathbf{a}_1 = \begin{bmatrix} 4 \\ 3 \end{bmatrix}, \qquad \mathbf{a}_2 = \begin{bmatrix} -2 \\ 1 \end{bmatrix}.$$

The first unit direction is

$$\mathbf{q}_1 = \frac{\mathbf{a}_1}{\|\mathbf{a}_1\|_2} = \frac{1}{5}\begin{bmatrix} 4 \\ 3 \end{bmatrix}.$$

Because $\mathbf{q}_1^\mathsf{T}\mathbf{a}_2 = (-8+3)/5 = -1$,

$$\mathrm{proj}_{\mathbf{q}_1}(\mathbf{a}_2) = -\mathbf{q}_1 = \begin{bmatrix} -\frac{4}{5} \\ -\frac{3}{5} \end{bmatrix}, \qquad \mathbf{u}_2 = \mathbf{a}_2 - \mathrm{proj}_{\mathbf{q}_1}(\mathbf{a}_2) = \begin{bmatrix} -\frac{6}{5} \\ \frac{8}{5} \end{bmatrix}.$$

Since $\|\mathbf{u}_2\|_2 = 2$, normalization gives

$$\mathbf{q}_2 = \frac{\mathbf{u}_2}{\|\mathbf{u}_2\|_2} = \begin{bmatrix} -\frac{3}{5} \\ \frac{4}{5} \end{bmatrix}.$$

Directly, $\mathbf{q}_1^\mathsf{T}\mathbf{q}_2 = (-12+12)/25 = 0$ and $\|\mathbf{q}_1\|_2 = \|\mathbf{q}_2\|_2 = 1$. The subtractions do not change the generated space, so

$$\mathrm{span}(\mathbf{a}_1, \ldots, \mathbf{a}_k) = \mathrm{span}(\mathbf{q}_1, \ldots, \mathbf{q}_k).$$

If at some iteration the orthogonalized vector $\widetilde{\mathbf{q}}_i = \mathbf{0}$, the corresponding input vector lies in the span of the previous vectors, so the input set is linearly dependent. Applied to traffic patterns, this tells the controller whether a proposed new pattern adds a genuinely new direction. Gram–Schmidt is preparation for QR decomposition and numerical linear algebra.

Check yourself. What operation is performed when we subtract $\mathrm{proj}_{\mathbf{u}_1}(\mathbf{v}_2)$ from $\mathbf{v}_2$?
Answer. We remove the component of $\mathbf{v}_2$ along $\mathbf{u}_1$, leaving a vector orthogonal to $\mathbf{u}_1$.

6 Vector spaces and subspaces

A *vector space* is a collection of objects for which vector addition and scalar multiplication behave consistently and remain in the collection. More formally, it is closed under those operations, has a zero vector and additive inverses, and addition and scalar multiplication satisfy associativity, commutativity of addition, identity, and the distributive and compatibility laws. The primary example is $\mathbb{R}^n$.

The physically realizable traffic states lie in $\mathbb{R}^4_{\geq 0}$, not all of $\mathbb{R}^4$. This is not a vector space over $\mathbb{R}$: multiplying a nonnegative queue state by -1 leaves the set. The ambient mathematical space remains $\mathbb{R}^4$, while physical constraints select only a subset.

A nonempty subset $S \subseteq V$ is a *subspace* if it is closed under addition and scalar multiplication. Equivalently,

$$\alpha\mathbf{u} + \beta\mathbf{v} \in S \quad \text{for all } \mathbf{u}, \mathbf{v} \in S \text{ and } \alpha, \beta \in \mathbb{R}.$$

In $\mathbb{R}^2$, a line through the origin is a subspace: sums and scalar multiples remain on the line. A line not through the origin is not: it omits $\mathbf{0}$. A plane through the origin in $\mathbb{R}^3$ is a subspace by the same closure argument. The first quadrant is not: multiplying a point by -1 leaves it. The union of a line and a plane is generally not: adding a point on the line to one on the plane can leave the union (unless the line is already contained in the plane).

Check yourself. Is a line in $\mathbb{R}^2$ that does not pass through the origin a subspace? Which condition fails?
Answer. No. It does not contain the zero vector. Equivalently, it fails closure under scalar multiplication, since multiplying any point by 0 produces $\mathbf{0}$.

Span and subspace are linked: $\mathrm{span}(\mathbf{v}_1, \ldots, \mathbf{v}_k)$ is always a subspace. A linear combination of vectors that are themselves linear combinations of the same generators is again a linear combination of those generators. Once a subset is a vector space, we can ask how many independent directions describe it: that number is its dimension.

7 Putting the Ideas Together

$$\text{data} \longrightarrow \text{vector} \longrightarrow \begin{cases} \text{combine: linear combinations,} \\ \text{measure: norms,} \\ \text{compare: distances and angles,} \\ \text{decompose: projections,} \\ \text{represent: span and basis,} \\ \text{simplify coordinates: an orthonormal basis.} \end{cases}$$

For the intersection, one vector represents one observed traffic state. For AI, the same chain describes one observation represented by features. A data set containing many observations is naturally organized as a matrix, which is the next step in the course.

Further reading. The material connects to the vector chapters of Boyd and Vandenberghe, *Introduction to Applied Linear Algebra*, and to the linear algebra chapters of Deisenroth, Faisal, and Ong, *Mathematics for Machine Learning*.